Session-Based Recommendation System Comparison

Transformers4Rec (XLNet) vs XGBoost Baselines

Next-Item Prediction on REES46 E-Commerce Session Data

March 17, 2026

Experiment Overview

Objective: Next-item prediction in e-commerce sessions. Given a user's browsing sequence (up to 19 interactions), predict the next product they will engage with.

Shared Data: All three models use the exact same training data and feature set (12 sequential features: product ID, category, subcategory, brand, price, recency, temporal signals). T4R architecture/hyperparameters were tuned by the original author; XGB models use default, untuned hyperparameters.

Evaluation Methodology: T4R uses incremental evaluation — train on day k, evaluate on day k+1 test data, for 30 days. XGB models use a standard train/test split — train on days 1-30, evaluate on day 31 only. For fair comparison, T4R's day 31 test results are used for comparison.

Metrics: NDCG@20, NDCG@40, Recall@20, Recall@40

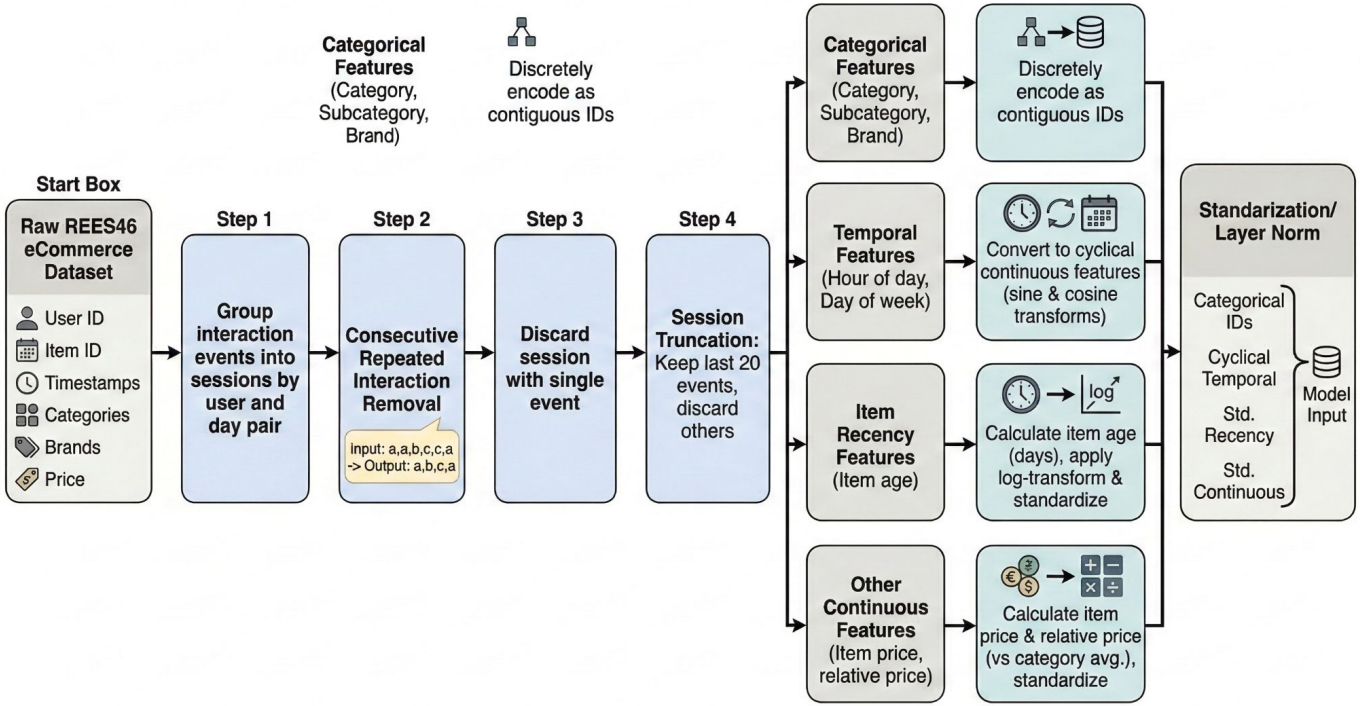
300K training sessions • ~390K unique products • Sequence length up to 20

Data Preprocessing

Input Features

Categorical (4):
- sess_pid_seq (item_id, 390K)
- sess_ccid_seq (category, 150)
- sess_csid_seq (subcategory, 1.4K)
- sess_bid_seq (brand, 7K)

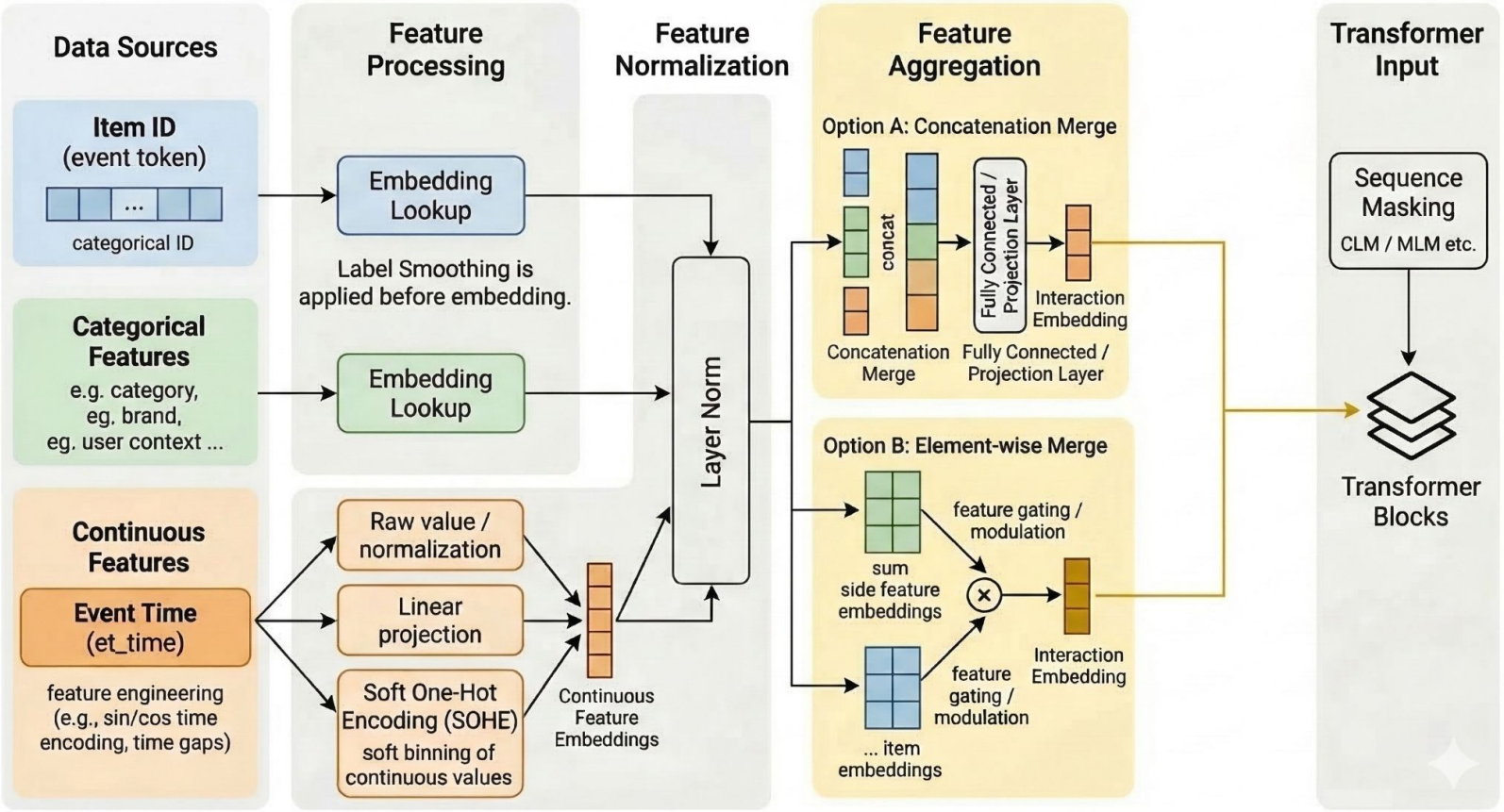
Continuous (7):
- price_log_norm
- relative_price_to_avg_category
- prod_recency_days_log_norm
- hour_sin, hour_cos
- dayofweek_sin, dayofweek_cos



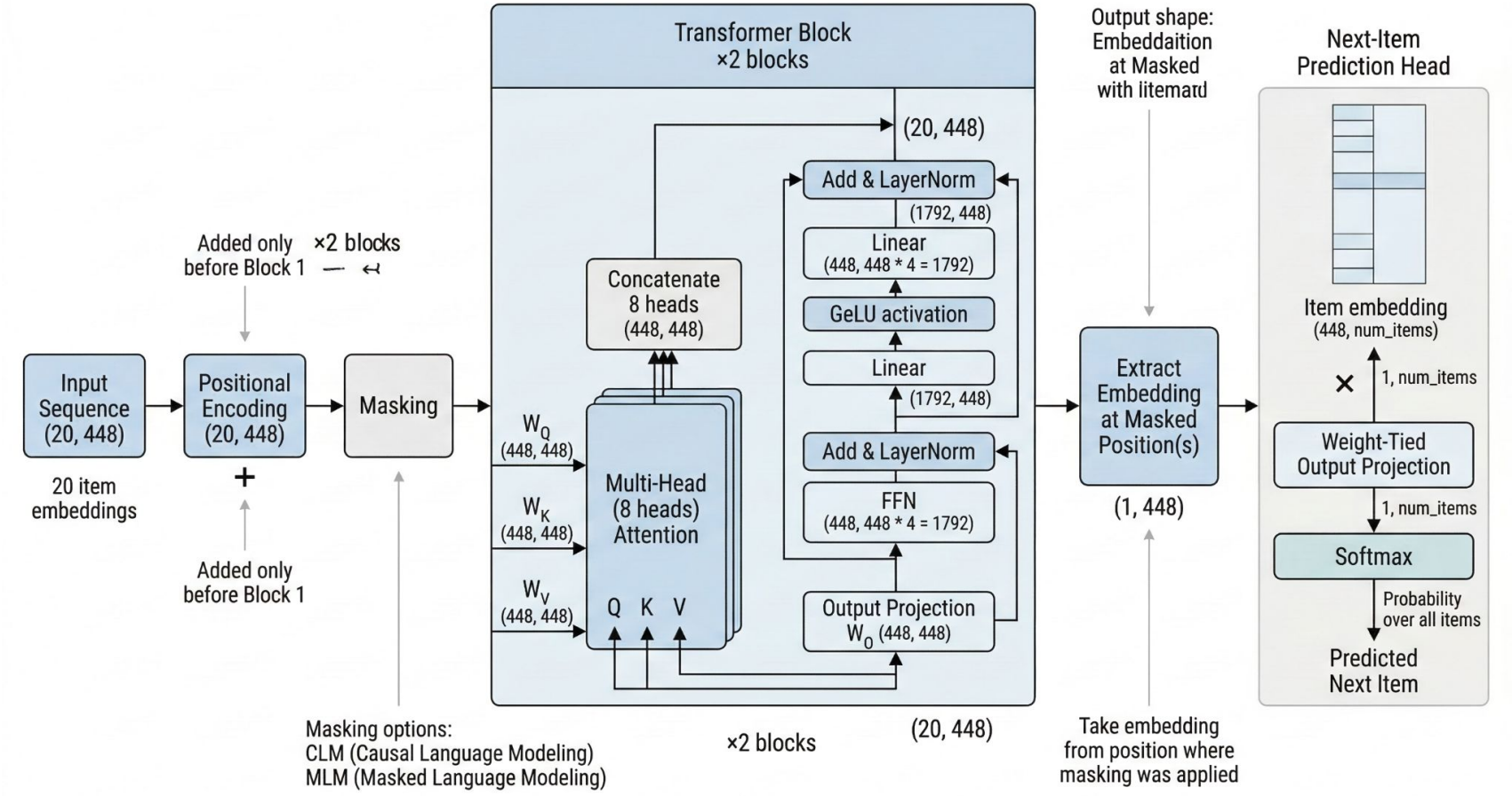
Model 1: Transformers4Rec (XLNet)

XLNet-MLM transformer with 2 layers, 8 attention heads, d_model=448. Uses masked language modeling (MLM) for next-item prediction. Trained incrementally via daily fine-tuning over 30 days.

T4R — Architecture: Embedding and Aggregation



T4R — Architecture: Transformer

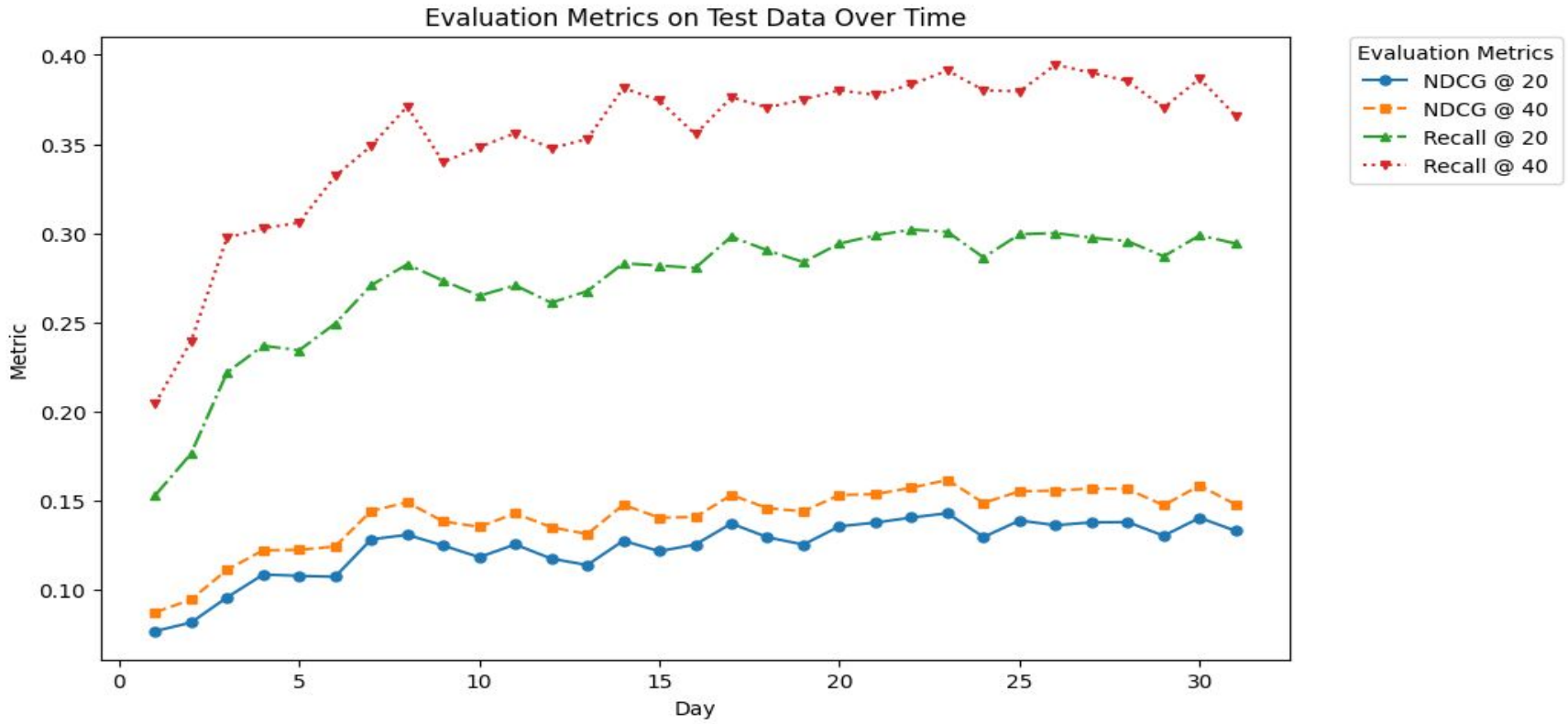


T4R — Hyperparameter Configuration

Parameter	Value	Category
d_model	448	Architecture
n_head	8	Architecture
n_layer	2	Architecture
hidden_act	gelu	Architecture
attn_type	bi (bidirectional)	Architecture
dropout	0	Architecture
max_sequence_length	20	Data
masking	MLM (p=0.1)	Training
learning_rate	2.017e-4	Optimization
lr_scheduler	linear	Optimization
weight_decay	2.747e-5	Optimization
label_smoothing	0.5	Optimization
num_train_epochs	10	Training
per_device_train_batch_size	256	Training

** Architecture and hyperparameters tuned by the original T4R author*

T4R — Incremental Evaluation Over 30 Days



Daily fine-tuning: train on day k, evaluate on day k+1 test data

T4R — Incremental Evaluation Trend Analysis

	NDCG@20	NDCG@40	Recall@20	Recall@40
Early Phase (Days 1-5)	0.0942	0.1077	0.2046	0.2700
Late Phase (Days 27-31)	0.1360	0.1535	0.2947	0.3797
Improvement	**+44.3%**	**+42.6%**	**+44.0%**	**+40.6%**
Overall Average (Days 1-31)	0.1241	0.1408	0.2722	0.3537

Rapid early learning: NDCG@20 grows from 0.077 (day 1) to 0.108 (day 4), a 41% improvement in just 4 days of incremental training.

Stable plateau after day 10: NDCG@20 mean = 0.1311 ± 0.0081, Recall@20 mean = 0.2881 ± 0.0123. Low variance indicates consistent performance.

Peak performance: Day 23 achieves the highest NDCG@20 = 0.1431 and Recall@40 = 0.3913.

T4R — Day 31 Test Results (Fair Comparison Point)

0.1331	0.1479	0.2943	0.3659
NDCG@20	NDCG@40	Recall@20	Recall@40

Day 31 test results used for apples-to-apples comparison with XGB models (also evaluated on day 31).

Evaluated on day 31 test partition • Model trained incrementally through day 30

T4R — Key Takeaways

- Incremental daily fine-tuning shows clear learning curve: metrics improve ~40-80% from day 1 to convergence around day 7-10.

- After convergence, performance is stable with low variance (NDCG@20 std = 0.008), suggesting robust generalization across days.

- The XLNet bidirectional attention (attn_type='bi') enables the model to leverage both past and future context in the session.

- Label smoothing (0.5) is unusually high — likely critical for the ~390K-class softmax to avoid overconfident predictions.

- Weight tying with a 448→64 projection reduces parameters significantly while maintaining prediction quality.

Model 2: XGBoost Baseline

2-model beam search: M1 predicts pid_1 (pid // 300), M2 predicts pid_2 (pid % 300) conditioned on pid_1. Top 10 x 10 = 100 candidates ranked by joint probability P(pid_1) x P(pid_2 | pid_1).

XGB Baseline — Configuration

Parameter	Value
`objective`	multi:softmax
`tree_method`	hist
`max_depth`	6 (default)
`learning_rate`	0.1 (default)
`n_estimators`	100 (default)
`device`	CUDA (training) / CPU (inference)

Prediction Strategy

Flat split: Encoded PID is divided into pid_1 (quotient ÷ 300) and pid_2 (remainder % 300).

M1: 228 features ⟶ 174 classes

M2: 229 features ⟶ 300 classes

All hyperparameters are XGBoost defaults (untuned).

XGB Baseline — Day 31 Results

0.1287	0.1353	0.2455	0.2775
NDCG@20	NDCG@40	Recall@20	Recall@40

Evaluated on 2,000 test samples from day 31 partition. 0 prediction failures.

XGB Baseline — Key Takeaways

- The flat pid split (quotient/remainder by 300) creates an arbitrary decomposition with no semantic meaning, limiting model effectiveness.

- Despite default hyperparameters, achieves NDCG@20 = 0.1287, providing a reasonable baseline for the task.

- Recall@40 (0.2775) is notably lower than T4R (0.3659), suggesting the beam search with only 100 candidates struggles to surface relevant items from ~52K unique products.

- Inference is slow (~1066s for 2K samples) due to row-by-row beam search with separate predict_proba calls for each pid_1 candidate.

Model 3: XGBoost Layered (Hierarchical)

4-model cascade exploiting the product category hierarchy: ccid → csid → pid_1 → pid_2. Each model conditions on predictions from the previous layer, encoding subcategories locally within their parent.

XGB Layered — Configuration

Parameter	Value
`objective`	multi:softmax
`max_depth`	6 (default)
`learning_rate`	0.1 (default)
`n_estimators`	100 (default)

4-Model Cascade Architecture

M1: base features → ccid (126 classes)

M2: base + ccid → csid (357 local classes)

M3: base + ccid + csid → pid_1 (43 local classes)

M4: base + ccid + csid + pid_1 → pid_2 (100 local classes)

Key Innovation: Local encoding re-indexes subcategories within their parent category. This dramatically reduces class counts:

- Raw pid: 52,049 unique values
- After local encoding: max 4,279 within any subcategory
- After pid split: 43 x 100 = manageable class sizes

XGB Layered — Day 31 Results

0.1372	0.1404	0.2635	0.2790
NDCG@20	NDCG@40	Recall@20	Recall@40

Evaluated on 2,000 test samples from day 31 partition. 0 prediction failures.

XGB Layered — Key Takeaways

- Hierarchical decomposition outperforms the flat baseline: NDCG@20 improves from 0.1287 to 0.1372 (+6.6%), Recall@20 from 0.2455 to 0.2635 (+7.3%).

- Exploiting the natural category hierarchy (ccid → csid → pid) provides meaningful structure that helps XGBoost make better predictions.

- Local encoding reduces effective class sizes dramatically (52K → max 4.3K per subcategory), making each XGBoost model's classification task more tractable.

- Despite the improvement, Recall@40 barely changes (0.2775 → 0.2790), suggesting the cascade's error propagation limits how many additional relevant items can be surfaced.

Model Comparison

T4R vs XGB Baseline vs XGB Layered on Day 31 test data. All models use the same features and training data.

Comparison — NDCG@20

Transformers4Rec	XGB Baseline	XGB Layered
0.1331	0.1287	0.1372
	-3.3% vs T4R	BEST
		3.1% vs T4R

NDCG@20 • Day 31 test data

Comparison — NDCG@40

Transformers4Rec	XGB Baseline	XGB Layered
0.1479	0.1353	0.1404
BEST	-8.5% vs T4R	-5.1% vs T4R

NDCG@40 • Day 31 test data

Comparison — Recall@20

Transformers4Rec	XGB Baseline	XGB Layered
0.2943	0.2455	0.2635
BEST	-16.6% vs T4R	-10.5% vs T4R

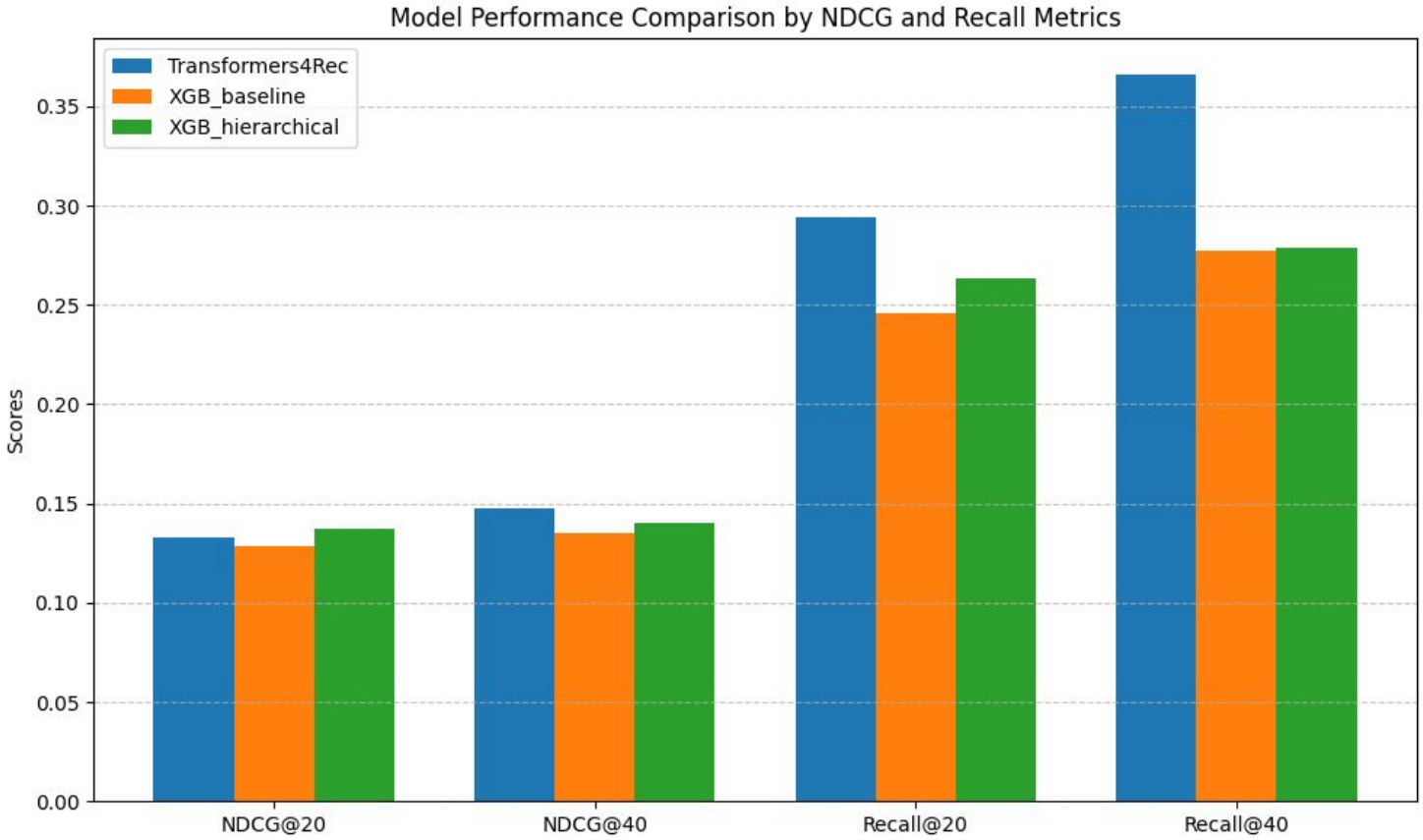
Recall@20 • Day 31 test data

Comparison — Recall@40

Transformers4Rec	XGB Baseline	XGB Layered
0.3659	0.2775	0.2790
BEST	-24.2% vs T4R	-23.7% vs T4R

Recall@40 • Day 31 test data

Comparison: Overall



Summary & Recommendations

Metric	T4R (Day 31)	XGB Baseline	XGB Layered
NDCG@20	0.1331	0.1287	0.1372
NDCG@40	0.1479	0.1353	0.1404
Recall@20	0.2943	0.2455	0.2635
Recall@40	0.3659	0.2775	0.2790

- T4R leads on all metrics, especially Recall: +19.9% Recall@20 and +31.9% Recall@40 vs XGB Baseline, demonstrating transformer's ability to capture sequential patterns.

- XGB Layered's hierarchical decomposition improves over Baseline (+6.6% NDCG@20, +7.3% Recall@20), validating that category structure helps.

- NDCG@20 gap is smaller (T4R 0.1331 vs XGB Layered 0.1372 = just 3%), suggesting XGB ranks its top predictions well, but T4R surfaces more relevant items overall.

- Recommendation: T4R is the stronger model. Next steps: tune XGB hyperparameters for fairer comparison; explore T4R with longer sequences and more training epochs.